TECHNICAL ARCHITECTURE

KamiCode Tech Stack

Startup-grade architecture, execution engine, AI evaluation layer, and AMD-compatible compute strategy.

AI-Native Evaluation | Developer Skill Graph | Verified Credentials | Startup Build Plan

Prepared for KamiCode - personal project and startup planning

Technical Stack and Architecture Document

This document defines the recommended practical stack for building KamiCode as a long-term personal project and startup-grade platform. It prioritizes speed of development first, then security, scale, AI capability, and optional AMD-compatible compute optimization.

1. Architecture Principles

• Build a reliable core coding loop before adding blockchain or hiring features.

• Use deterministic execution for correctness and AI only for reasoning and feedback.

• Keep services modular so execution, AI, scoring, and credentials can scale independently later.

• Start with managed services where helpful; move to self-hosted compute when traffic or cost requires it.

• Design for AMD CPU/GPU compatibility by keeping AI workloads compatible with ROCm or CPU fallback.

2. Recommended Stack

Layer	Recommended Choice	Reason
Frontend	Next.js + TypeScript	Modern full-stack React framework with strong ecosystem and scalable code structure.
UI	Tailwind CSS + shadcn/ui	Fast, polished interface for a developer-focused aesthetic.
Code Editor	Monaco Editor	VS Code-like coding experience in browser.
Backend API	FastAPI + Python	Excellent for API, AI integration, async jobs, and clean documentation.
Database	PostgreSQL	Reliable relational storage for users, problems, submissions, stats, and achievements.
Cache/Queue	Redis	Leaderboards, rate limiting, queues, and fast session data.
Execution	Docker sandbox workers	Practical first version for running user code with limits.
AI Layer	OpenAI/Anthropic first; self-hosted later	Move fast first, then optimize for cost and control.
Blockchain	Polygon/Base + Solidity later	Optional credential layer after skill graph is useful.
Storage	S3-compatible / Supabase Storage	Problem assets, metadata, logs, and credential JSON.

3. AMD-Compatible Compute Strategy

• Execution workers benefit from AMD EPYC-style high core count CPUs because many submissions can be evaluated in parallel.

• AI inference can later use AMD GPUs through ROCm-compatible PyTorch or ONNX Runtime paths.

• The platform should support CPU fallback so development and deployment are not blocked by GPU availability.

• Self-hosted coding models should be introduced after traffic is consistent and AI cost becomes meaningful.

4. Core Service Breakdown

Auth Service

GitHub/email login, user identity, wallet address linking later.

Problem Service

Problem bank, tags, difficulty, examples, hidden test cases.

Submission Service

Receives code, creates submission record, queues execution.

Execution Workers

Run code in isolated Docker containers with time/memory limits.

AI Analysis Service

Analyzes passed submissions for complexity, approach, readability, and improvement advice.

Scoring Engine

Combines correctness, runtime percentile, and AI quality score.

Skill Graph Service

Updates topic mastery and developer profile stats.

Achievement Service

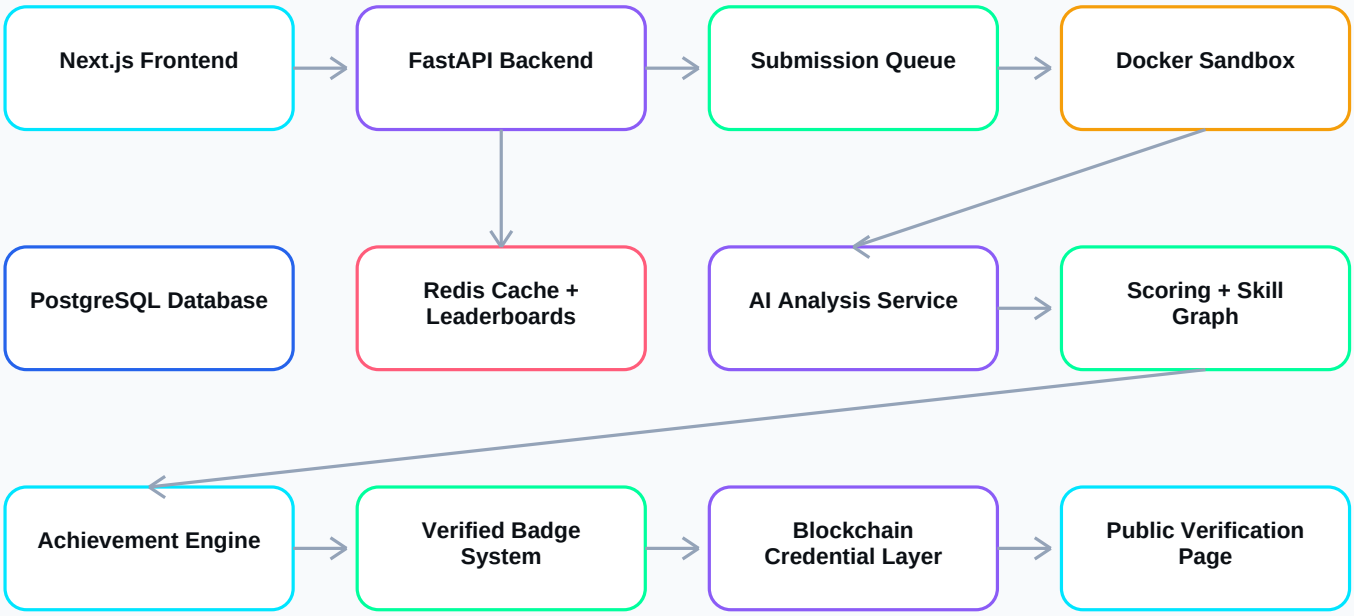
Checks rules for badges, streaks, rankings, and verified credentials.

Credential Service

Optional blockchain minting and verification pages.

5. System Architecture Flowchart

System Architecture - Startup Version



Core rule: deterministic test execution decides correctness; AI evaluates complexity, approach quality, and improvement guidance.

6. Code Execution Flow

Execution Worker Flow

7. Database Schema - First Version

Table	Important Fields
users	id, name, username, email, github_id, wallet_address, created_at
problems	id, title, slug, description, difficulty, topic_tags, constraints, examples
test_cases	id, problem_id, input, expected_output, is_hidden
submissions	id, user_id, problem_id, language, code, status, runtime_ms, memory_kb, passed_tests, total_tests
ai_analysis	id, submission_id, time_complexity, space_complexity, approach_detected, feedback, score
achievements	id, user_id, submission_id, type, title, is_verified, is_minted, token_id
skill_stats	id, user_id, topic, attempted, solved, average_ai_score, mastery_score

8. API Contract - Starter Set

Endpoint	Purpose
GET /problems/today	Returns today's active problem.
POST /submissions	Creates and queues a code submission.
GET /submissions/{id}	Returns execution result and AI analysis.

GET /leaderboard/daily	Returns daily ranked submissions.
GET /profiles/{username}	Returns public skill profile.
POST /achievements/{id}/mint	Optional credential mint endpoint after verification.

9. Deployment Plan

• Development: local Next.js, local FastAPI, Docker Desktop, PostgreSQL, Redis.

• Private alpha: single VPS with Docker Compose, managed PostgreSQL, object storage.

• Growth stage: separate execution worker nodes, Redis queue, managed database, autoscaling backend.

• Scale stage: Kubernetes, isolated execution cluster, dedicated AI inference service, observability stack.

10. Security Checklist

• Run user code only in isolated containers or stronger sandboxing.

• Disable network access inside execution containers.

• Enforce CPU, memory, process, and file limits.

• Store hidden test cases securely and never expose them to frontend.

• Rate limit submissions and auth endpoints.

• Perform server-side validation before achievements or credentials are issued.